

Ingeniería en Ciberseguridad e Infraestructura de Cómputo

Programación en Red
Batalla Naval (TCP y UDP)

Brayan Usciel Seseña Hidalgo



Universidad Veracruzana

Batalla naval en TCP

Defensor (defensor.py)

```
✓ [11:31 ] lonerism@loreta:~/prog_red/batalla_naval 10.30.3.103 59% $ python3 de
fensor.py
Defensor esperando en puerto 5050...
Atacante conectado desde ('10.30.3.103', 56816)
Ataque: A1 | Evaluación: TOCADO | Barcos restantes: 3
Ataque: A2 | Evaluación: TOCADO | Barcos restantes: 2
Ataque: B3 | Evaluación: TOCADO | Barcos restantes: 1
Ataque: C4 | Evaluación: TOCADO | Barcos restantes: 0
□
```

Atacante (atacante.py)

```
✓ [11:30 ] lonerism@loreta:~/prog_red/batalla_naval 10.30.3.103 59% $ python3 at
acante.py
Ingresar coordenada de ataque (ej. B4): A1
Respuesta del defensor: TOCADO
Ingresar coordenada de ataque (ej. B4): A2
Respuesta del defensor: TOCADO
Ingresar coordenada de ataque (ej. B4): B3
Respuesta del defensor: TOCADO
Ingresar coordenada de ataque (ej. B4): C4
Respuesta del defensor: TOCADO
Ingresar coordenada de ataque (ej. B4): A2
Respuesta del defensor: AGUA
Ingresar coordenada de ataque (ej. B4): □
```

Programa TCP en Python

Todos los programas hacen uso de la librería socket.

Defensor (servidor)

En el defensor se crea un arreglo que almacenara las coordenadas de los barcos.

En el programa se especifica la IP del servidor y su puerto.

El servidor se pone en modo escucha en el puerto 5050.

El servidor decodifica las coordenadas enviadas en UTF-8 desde el atacante (cliente).

Si la coordenada recibida es una de las que esta en el arreglo se envía un mensaje de “Tocado” o “Agua” si no lo alcanza.

Atacante (cliente)

Se crea el socket del cliente.

Se especifica la IP a la que se va a conectar (servidor) y su puerto para formar el socket del servidor.

Con un bucle while se envían las coordenadas codificadas hacia el servidor.

Se recibe la respuesta de la coordenada enviada hacia el servidor decodificándola para saber si se toco un barco o no.

Batalla naval el UDP

Defensor (defensor_udp.py)

```
✓ [11:35 ] lonerism@loreta:~/prog_red/batalla_naval 10.30.3.103 57% $ python3 de
fensor_udp.py
Defensor UDP activo en 5050. Esperando datagramas...
Ataque: A1 desde ('10.30.3.103', 33688) | Evaluación: TOCADO | Barcos restantes:
3
Ataque: A2 desde ('10.30.3.103', 33688) | Evaluación: TOCADO | Barcos restantes:
2
Ataque: B3 desde ('10.30.3.103', 33688) | Evaluación: TOCADO | Barcos restantes:
1
Ataque: C4 desde ('10.30.3.103', 33688) | Evaluación: TOCADO | Barcos restantes:
0
Ataque: C1 desde ('10.30.3.103', 33688) | Evaluación: AGUA | Barcos restantes: 0
□
```

Atacante (atacante_udp.py)

```
✓ [11:35 ] lonerism@loreta:~/prog_red/batalla_naval 10.30.3.103 57% $ python3 at
acante_udp.py
Coordenada de disparo: A1
Respuesta recibida: TOCADO
Coordenada de disparo: A2
Respuesta recibida: TOCADO
Coordenada de disparo: B3
Respuesta recibida: TOCADO
Coordenada de disparo: C4
Respuesta recibida: TOCADO
Coordenada de disparo: C1
Respuesta recibida: AGUA
Coordenada de disparo: □
```

Programa TCP en Python

Los programas en UDP tambien usan la librería socket.

Defensor (servidor)

En el programa se especifica principalmente el puerto del servidor.

El servidor se pone en modo escucha en el puerto 5050.

El servidor decodifica las coordenadas enviadas en UTF-8 desde el atacante (cliente).

Si la coordenada recibida es una de las que esta en el arreglo se envía un mensaje de “Tocado” o “Agua” si no lo alcanza.

Atacante (cliente)

Se crea el socket del cliente.

Se especifica la IP a la que se va a conectar (servidor) y su puerto para formar el socket del servidor al que se conectara.

Con un bucle while se envían las coordenadas codificadas hacia el servidor.

Se recibe la respuesta de la coordenada enviada hacia el servidor decodificándola para saber si se toco un barco o no.

Diferencia entre TCP y UDP

La principal diferencia entre ambas batallas, es similar a como difieren ambos protocolos, donde TCP si se asegura mas de la conexión, mientras que UDP no se asegura de la misma forma.